Numerical Quadrature

Central Question: How can we approx. an integral accurately using only a small number of function evaluations?

Why do we need numerical integration?

We want to calculate

$$I = \int_a^b f(x)\,dx \qquad f:[a,b] \to \mathbb{R}$$

from calculus, we know that if an antiderivative F is available,

$$I = F(b) - F(a)$$

But some do not have F or f might represent experimental data or a complicated numerical model!

we evaluate f at selected points and use those to estimate the total integral

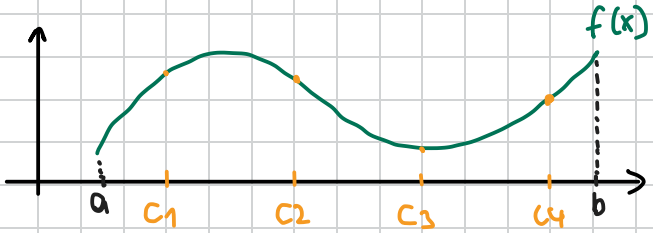
Def 5.3.1 Quadrature

index, not power ⇝ equivalently $w_{i,n}$ and $c_{i,n}$

$$Q_n(f;a,b) := \sum_{i=1}^{n} w_i^{(n)} f(c_i^{(n)}) \approx \int_a^b f(x)\,dx$$

$w_i^{(n)}$: weights; $c_i^{(n)} \in [a,b]$: nodes

Intuition: Each $f(c_i^{(n)})$ tells the height of the function at $c_i^{(n)}$ and $w_i^{(n)}$ determines how strongly that sampled height contributes to the estimated area



Both weights and nodes are important as: Evaluating a function at wrong locations may miss important behavior. Choosing inappropriate weights can make even excellent sample locations give the wrong area.

Definition 5.3.3 Error

algebraic and exponential convergence

$$E(n) = \int_a^b f(n)\,dx - Q_n(f; a, b)$$

When comparing algorithms we usually use $|E(n)|$. We want to make this small as possible. Usually with $n \to \infty \Rightarrow E(n) \to \infty$.

Constructing quadrature from interpolation

We want to replace f by another function whose integral is easy.

⇒ Consider replacing f with an interpolation polynomial

$$\int_a^b f(x)\,dx \approx \int_a^b p_n(x)\,dx \qquad p_n = \alpha_0 + \alpha_1 x + \dots + \alpha_n x^n$$

This results in Vandermonde!

Reminder – Lagrange Polynoms

For distinct interp nodes $x_0, \dots, x_n$: $l_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$

$$l_i(x_j) \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} \Rightarrow p_n(x) = \sum_{i=0}^{n} f(x_i)\, l_i(x)$$

$$\Rightarrow \int_a^b f(x)\,dx \approx \int_a^b p_n(x)\,dx = \int_a^b \sum_{i=0}^{n} f(x_i)\, l_i(x)\,dx$$

$$\int_a^b \sum_{i=0}^{n} l_i(x)\,dx = \sum_{i=0}^{n} \int_a^b l_i(x)\,dx \quad \text{with } f(x_i) \text{ being constant.}$$

$$= \sum_{i=0}^{n} f(x_i) \int_a^b l_i(x)\,dx$$

$f(x_i) = c_i$, $\int_a^b l_i(x)\,dx = w_i$

Nodes are interpolation nodes and weight are integrals of corresponding basis functions.

Quadrature Ordnung

Script states: Quad. order = 1 + highest dep. poly integrated exactly.

→ So if a rule integrates all polynomials up to degree n exactly, it has the order n+1. (*)

guarantees are only for polynomials

Why we test polynomials?

- Because every poly. is a linear combination of monomials:

$$p(x) = a_0 + a_1 x + \cdots + a_m x^m$$

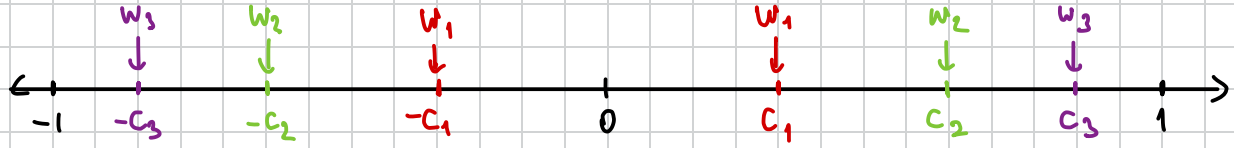
and both quadrature and integration are linear operations. If the rule is exact for $1, x, x^2, \ldots, x^m$ then it is exact for every linear combination of these.

Extra: On $[-1,1]$ for $f(x) = x^k$, $\int_{-1}^{1} x^k \, dx = \begin{cases} 0 & k \text{ odd} \\ \frac{2}{k+1} & k \text{ even} \end{cases}$

Def 5.3.9 Symmetric Quadraturformel

A quadrature rule on $[-1,1]$ is symmetric if:

$$w_i = w_{n+1-i} \quad \text{and} \quad c_i = -c_{n+1-i}$$



Theorem 5.3.11 The order of symmetric quadrature rule is even.

Why?

→ Suppose we have two nodes, $-c$ and c with the same weight. Their contri. to the quadrature is $wf(-c) + wf(c)$

→ Now let f be an odd function, like x^3. For every odd function we have $f(-c) = -f(c) \Rightarrow wf(-c) - wf(c) = 0$

→ Since we have symmetric nodes this happens for every pair $\Rightarrow Q(f) = 0$

→ But observe that $\int_{-1}^{1} f(x)\,dx = 0$, so our quadrature is exact for every odd f.

! As we are testing monomials for a rule the odd powered monomials are given as they are odd

Classical Quadrature

Midpoint Rule

$$Q^M(f; a, b) = (b-a)\, f\left(\frac{a+b}{2}\right)$$

Intuition: We evaluate the function once at the middle of the interval and pretend the entire function has that constant height.



Order: On $[-1, 1]$, we test $1, t, t^2, \dots$

$\rightarrow Q^M(f) = (1-(-1))\, f\left(\frac{-1+1}{2}\right) = 2f(0)$

Test deg. 0: $I(1) = \int_{-1}^{1} 1 \, dt = 2$, $Q^M(1) = 2(1) = 2$ ✓

Test deg. 1: $I(t) = \int_{-1}^{1} t \, dt = 0$, $Q^M(t) = 2(0) = 0$ ✓

Test deg. 2: $I(t^2) = \int_{-1}^{1} t^2 \, dt = \frac{2}{3}$, $Q^M(t^2) = 2(0^2) = 0$ ✗

$\Rightarrow$ Order is 2

! Here interpolation poly. is a constant (i.e $f(\frac{a+b}{2})$) but order is because of symmetry.

Trapezoidal Rule

$$Q^T(f; a, b) = \frac{b-a}{2}\left(f(a) + f(b)\right)$$

Intuition: We take the two endpoint heights. Connect them with a straight line and integrate underneath it.



Observe that order is 2 as expected by (★)

Simpson Rule

$$Q^S(f;a,b) = \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right]$$

Intuition: Rather than constructing them with a line we construct a degree-2 polynomial.

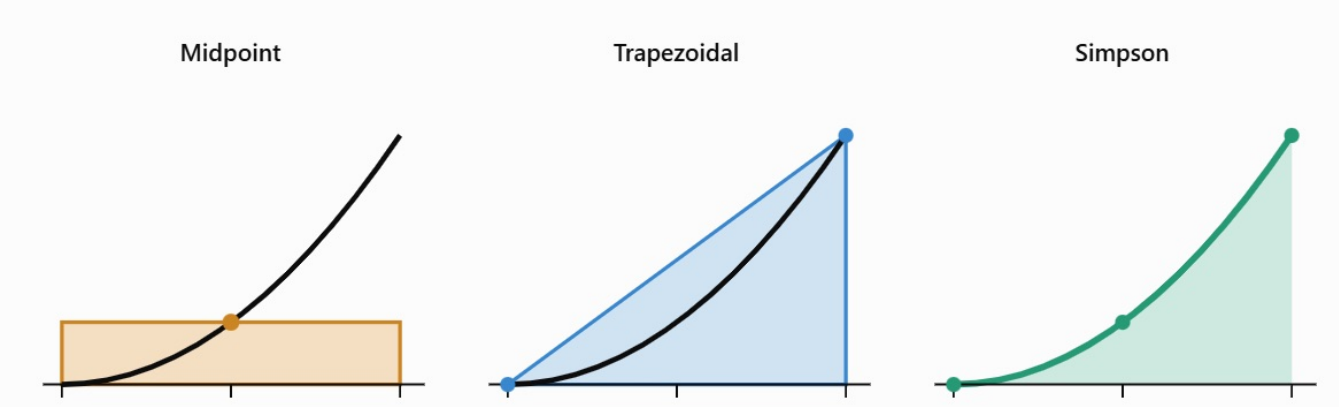


Order: We test on $[-1, 1]$ $\Rightarrow$ so we have 3 nodes $t = -1, 0, 1$.

$$Q^S(f) = \frac{1}{3}\left[f(-1) + 4f(0) + f(1)\right]$$

Test. deg 0:	$I(1) = 2$	$Q^S(1) = \frac{1+4+1}{3} = 2$	✓
Test deg 1:	$I(t) = 0$	$Q^S(t) = 0$	by symmetry
Test deg. 2:	$I(t^2) = \frac{2}{3}$	$Q^S(t^2) = \frac{(-1)^2 + 4(0^2) + 1^2}{3} = \frac{2}{3}$	✓
Test deg. 3:	$I(t^3) = 0$	$Q^S(t^3) = 0$	by symmetry
Test deg 4:	$I(t^4) = \frac{2}{5}$	$Q^S(t^4) = \frac{(-1)^4 + 4(0^4) + 1^4}{3} = \frac{2}{3}$	✗

$\Rightarrow$ Order is 4 ... we also see order is the first failure degree!



Composite Quadrature

Suppose the integration interval $[a,b]$ is too large for one accurate local approximation. Lets diviole it into N equal subintervals

$h = \frac{b-a}{N}$ ... width of each piece

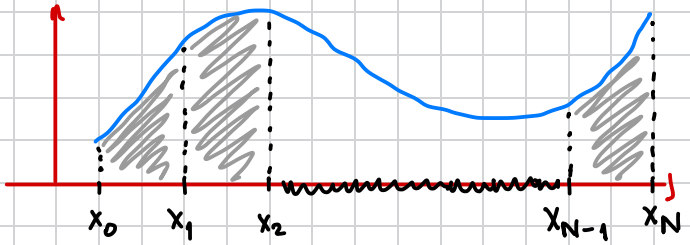
Starting at a we move right by one step of length h, then another, so on:

$x_i = a + ih \quad i = 0, \dots, N$ ... these are the nodes

Observe $x_0 = a$ and $x_N = a + N\left(\frac{b-a}{N}\right) = b$

$\Rightarrow [x_0, x_1], [x_1, x_2], \dots, [x_{N-1}, x_N]$

This is useful for integration as



$$\Rightarrow \int_a^b f(x)\,dx = \sum_{i=0}^{N-1} \int_{x_i}^{x_{i+1}} f(x)\,dx$$

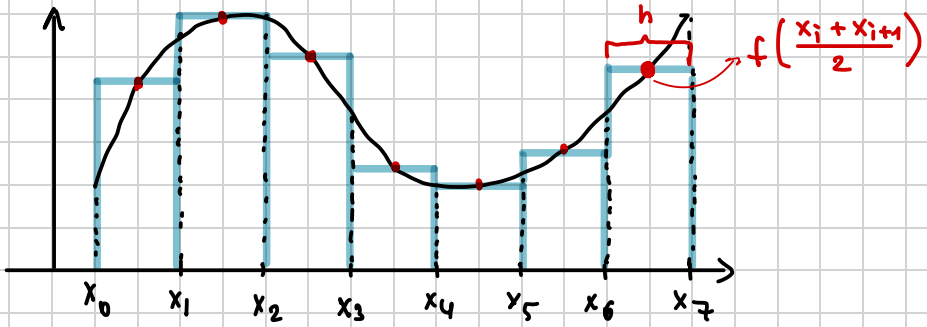
Now we can approximate each small integral with a simple rule and simply add the results.

$$\int_a^b f(x)\,dx \approx \sum_{i=0}^{N-1} Q(f; x_i, x_{i+1})$$

Increasing N will decrease h, making the grid finer $\Rightarrow$ Better approximation

Composite Midpoint Rule

On subinterval $[x_i, x_{i+1}]$ midpoint is $\frac{x_i + x_{i+1}}{2}$. We approx. the area by a rectangle with width h and midpoint height.



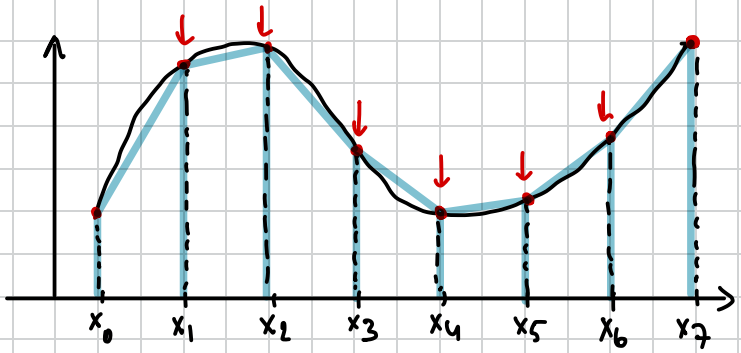
$$\Rightarrow I(f; a, b) = h \sum_{i=0}^{N-1} f\left(\frac{x_i + x_{i+1}}{2}\right)$$

Composite Trapezoidal Rule

On one subinterval

$$Q^T(f; x_i, x_{i+1}) = \frac{h}{2}\left[f(x_i) + f(x_{i+1})\right]$$

$$\Rightarrow \frac{h}{2} \sum_{i=0}^{N-1} \left[f(x_i) + f(x_{i+1})\right]$$



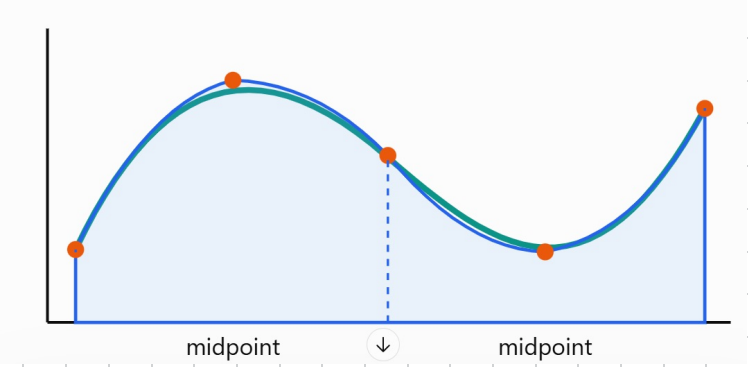
Notice every interior grid point occurs twice only a and b occur once in the sum

$$\Rightarrow I(f; a, b) \approx \frac{h}{2}\left[f(a) + 2\sum_{i=1}^{N-1} f(x_i) + f(b)\right]$$

x_1 to x_6 above

Composite Simpson Rule

On each subinterval we use a parabola passing through its two endpoints and midpoint



On every subinterval we have

$$\frac{h}{6}\left[f(x_i) + 4f\left(\frac{x_i + x_{i+1}}{2}\right) + f(x_{i+1})\right]$$

$$\Rightarrow I(f; a, b) \approx \frac{h}{6} \sum_{i=0}^{N-1} \left[f(x_i) + 4f\left(\frac{x_i + x_{i+1}}{2}\right) + f(x_{i+1})\right]$$

Section: Error Altogether

Recall the quadrature error:

$$E(n) = \int_a^b f(n)\,dx - Q_n(f; a, b)$$

Ideally $E(n) \to 0$ as $n \to \infty$, but how quickly does it decrease?

Reminder - Convergence Rates

Algebraic: $O(N^{-p})$

Exponential: $O(q^N)$ $0 < q < 1$

We started with original function f, approximated it with a simpler interp. polynomial p, and integrated that polynomial:

$$Q(f) = \int_a^b p(x)\,dx$$

$$\Rightarrow \text{error is } \int_a^b f(x)\,dx - \int_a^b p(x)\,dx$$

$$\Leftrightarrow \int_a^b (f(x) - p(x))\,dx \quad \text{... Quadrature error is the integral of interpolation error}$$

Observe the magnitude:

$$\left| \int_a^b (f(x) - p(x))\,dx \right| \leq \int_a^b |f(x) - p(x)|\,dx \leq h \|f - p\|_\infty$$

So integration multiplies the max interp. error by the interval length

For composite rules

- Local error $O(h^{p+1})$ where p is order of the rule
  error made in one interval

$$e_i = \int_{x_i}^{x_{i+1}} f(x)\,dx - Q_i(f) \quad \text{and} \quad e_{global} = \sum_{i=0}^{N-1} e_i$$

$$\Rightarrow e_{global} \leq \sum_{i=0}^{N-1} \underbrace{|e_i|}_{\text{local error}} \leq NCh^{p+1} = \frac{b-a}{h} Ch^{p+1}$$

$\hookrightarrow h = \frac{b-a}{N}$

$$\Rightarrow e_{global} = O(h^p) = O(N^{-p}) \text{ ... algebraic convergence}$$

Reference Intervals

Composite quadrature introduces an inconvenience. We have many different subintervals. Would we need new quad. nodes and weights for each one?

→ No, instead we construct a quad rule on a reference interval and transform it.

→ Standard choice is $[-1, 1]$ → could be any interval

So say a reference node $\tau = -1 \rightarrow t = a$ and for $\tau = 1 \rightarrow t = b$

this changes

$$\Rightarrow t = \frac{1}{2}(1-\tau)a + \frac{1}{2}(1+\tau)b \quad \rightsquigarrow \text{just } \tau = \frac{2t-(a+b)}{b-a}$$

where $\tau \in [-1, 1]$ and $t \in [a, b]$

$$\Rightarrow 2t = \tau b - \tau a + a + b$$
$$2t = (1-\tau)a + (1+\tau)b$$
$$t = \frac{1}{2}(1-\tau)a + \frac{1}{2}(1+\tau)b$$

also this changes!

Check its endpoints ↑

! To get the nodes and weights:

$$dt = \frac{b-a}{2}\, d\tau \quad \Rightarrow \int_a^b f(t)\, dt = \frac{b-a}{2} \int_{-1}^{1} \hat{f}(\tau)\, d\tau$$

$\frac{dt}{d\tau} = \frac{b-a}{2}$

where $\hat{f}(\tau) = f\left(\frac{1}{2}(1-\tau)a + \frac{1}{2}(1+\tau)b\right)$

And if the reference nodes and weights are $\hat{c}_j$ and $\hat{w}_j$, their transformed counterparts are

$$c_j = \frac{1}{2}(1-\hat{c}_j)a + \frac{1}{2}(1+\hat{c}_j)b \quad , \quad w_j = \frac{b-a}{2}\hat{w}_j$$

Be careful this only holds for reference interval $[-1, 1]$!

In general let $[\alpha, \beta]$ be the ref interval → $t = a + \frac{b-a}{\beta-a}(\tau - \alpha)$

$$\Rightarrow c_j = a + \frac{b-a}{\beta-a}(\hat{c}_j - \alpha) \quad \text{and} \quad w_j = \frac{b-a}{\beta-\alpha}\hat{w}_j$$

This isn't in the script!

Code Expert: Unbekannte Quadraturformel

We are given a set of nodes and weights on $[-1,1]$, but the order of the quad. rule is unknown.

The task is to determine it, construct a composite version and study convergence on two integrands.

Coding Tips

- Use Def. 5.3.1
- for determining order test $1, x, x^2 \dots$, use np.isclose()
- for composite_Q you have to map the weight and nodes for every subinterval $[x_i, x_{i+1}]$. Look at the reference interval section, substitute $a = x_i$, $b = x_i + h \sim x_{i+1}$.

(x_n and w_n are def. on $[-1,1]$)

Better node locations: Gaussian Quadrature

We refined the intervals to get better accuracy, now we select the locations of the function evaluations we use!

Consider a quad. rule with s nodes on $[0,1]$: $Q(f;0,1) = \sum_{j=1}^{s} b_j f(c_j)$

Now we construct a poly. that vanishes at every point: $M(t) = \prod_{j=1}^{s} (t - c_j)$

$\Rightarrow M(c_j) = 0$ for all c_j.

$\Rightarrow$ Thus $Q(M;0,1) = 0$ regardless of the weights for c_j and for every poly. g:

$$Q(Mg;0,1) = 0 = \sum_{i=1}^{s} w_i M(t) g(t)$$

This means the quad rule cannot detect a poly. contribution containing M. But if we can choose M so that its integral against low-degree polynomials also vanishes, that invisible contri. won't produce an integration error.

↳ $M(t)g(t)$ might be non-zero between the nodes, but we cannot detect

Theorem 5.5.1

An interpolatory quad. rule has order $s+m$ exactly when $\langle M, f \rangle = \int_a^b M(t) f(t)\,dt = 0$ for every poly. f of degree at most $m-1$

↓

For some f of sufficiently high degree, by polynom division:

$$f(t) = M(t)g(t) + r(t)$$

where remainder r has degree at most $s-1$ (as $\deg(M) = s$).

The quadrature already integrates r exactly as we have s nodes!

We know at the quad. nodes $Mf = 0$ holds. Therefore the quad. res. for f is the same as r.

To make the original integral behave the same way, we need

$$\int_0^1 M(t)\,g(t)\,dt = 0. \quad \Big\} \text{ for no errors this must hold.}$$

⇒ So we must choose the nodes so that the part of polynomial f that the quad. cannot see also has zero total area.

Theorem 5.5.2

A quadrature formula with s nodes has order at most $2s$

≈ highest poly deg. we can hope to integrate exactly is $2s-1$.

A s-node Gaussian quad. rule has order $2s$

Theorem 5.5.3

There exists a unique sequence of monic polynomials $p_0, p_1, \ldots$ with p_k of degree k s.t.:

$$p_k \perp \operatorname{span}\{p_0, \ldots, p_{k-1}\}$$

These poly. satisfy a 3-term recurrence:

$$p_{k+1}(x) = (x - \beta_{k+1})\,p_k(x) - \gamma_k\,p_{k-1}(x)$$

This is telling us these exists $M(t) = p_k(t)$ s.t. it is orthogonal to all $g \in \operatorname{span}\{p_0, \ldots, p_{k-1}\}$ with $\deg(g) \le k-1$. So let $k = s$, we have what we wanted!

And as $M(t) = (t-c_1)(t-c_2)\cdots(t-c_k) = p_k(t)$ we have that c's are roots of $p_k(t)$

↓

Remark 5.5.7 ★★★

The Gaussian nodes are the roots of p_s

Example 5.5.4.

On interval $[-1,1]$ the orthogonal pdy. from Th. 5.5.3 are called Legendre polynomials

$$p_0(t)=1 \;;\; p_1(t)=t \;;\; p_2(t)=\frac{1}{2}(3t^2-1) \;;\; p_3(t)=\frac{1}{2}(5t^3-3t)$$

→ These Legendre pdy. are normalized s.t. $p_s(1)=1$, rather than being monic. They are therefore constant multiples of our M.

Example 5.5.9 Gaussian Quad. with $s=2$

❗ We need degree-2 $M(t)$ orthogonal to all poly. of degree at most 1.

Case 1: Interval $[1,0]$

$$M(t) = (t-c_1)(t-c_2) = t^2 + At + B$$

$$\Rightarrow \int_0^1 M(t)\,g(t)\,dt = 0 \quad \text{we test } g(t)=1 \text{ and } g(t)=t.$$

Integrate $\int_0^1 t^2+At+B\,dt = 0$ and $\int_0^1 (t^2+At+B)t\,dt = 0$, find A and B

$$\Rightarrow p_2(t) = M(t) = t^2 - t + \frac{1}{6}$$

⇒ Find $M(t)$'s roots, they are the nodes!

Case 2: Interval $[-1,1]$ ⇝ Example 5.5.4 applies

$$p_2(t)=\frac{1}{2}(3t^2-1) \text{ and } c_1=-\frac{1}{\sqrt{3}} \text{ and } c_2=\frac{1}{\sqrt{3}}$$

For weights use exactness to determine them: $Q(f) = w_1 f(c_1) + w_2 f(c_2)$

→ Test $f(t)=1$: $I(1)=2$; $w_1(1)+w_2(1)=2$

→ Test $f(t)=t$: $I(t)=0$; $w_1\left(-\frac{1}{\sqrt{3}}\right)+w_2\left(\frac{1}{\sqrt{3}}\right)=0$

] Solve the system!

Code Expert : Gauss-Legendre Quadratur

Coding tips

- scipy.special.roots_legendre(n) returns on [-1, 1] but gauss_legendre is on [left, right], so again refer to reference interval section!
- for composite rules you might find np.linspace() useful

Weight function

$$\langle M, g \rangle = \int_a^b M(x) g(x) \underline{w(x)}\, dx$$

not the weights in quad rule!

→ We will see different weights will give different orthogonal polynom families.

Radau Quadratur

Let $c_s = 1$, so we fix last node to be right endpoint on $[-1, 1]$, unlike Gauss-Legendre which chooses all its nodes inside the interval.

↳ We then select the other nodes in an optimized way.

> **Remark 5.5.11** For s nodes, Radau order is $2s-1$

↳ compared to Gauss Quad we lost an order! ~ because one degree of freedom is lost.

How are remaining Radau nodes constructed?

Recall $M(t) = \prod_{j=1}^{s} (t - c_j)$, we fix $c_s = 1 \Rightarrow M(t) = (t-1) \underbrace{\prod_{j=0}^{s-1} (t - c_j)}_{P(t)}$

$\Rightarrow$ we want $\int_{-1}^{1} M(t) g(t)\, dt = 0 \Leftrightarrow \int_{-1}^{1} (t-1) P(t) g(t)\, dt = 0$

again an orthogonality condition w.r.t $(t-1) = w(t)$!

$\times(-1)$

$\Rightarrow$ In practice $\int_{-1}^{1} (1-t) P(t) g(t)\, dt = 0$, the roots of the orthogonal poly is our remaining nodes

Jacobi Polynomials

The Jacobi weight function is $w(t) = (1-t)^{\alpha}(1+t)^{\beta}$ and different α, β values result in different orthogonal poly. families.

α	β	Weight function $\rho(t)$	Quadrature rule / application
0	0	1	Gauss–Legendre
1	0	$1 - t$	Gauss–Radau, right endpoint fixed at 1
0	1	$1 + t$	Gauss–Radau, left endpoint fixed at -1
1	1	$1 - t^2$	Gauss–Lobatto, both endpoints fixed
$-\frac{1}{2}$	$-\frac{1}{2}$	$\frac{1}{\sqrt{1-t^2}}$	Gauss–Chebyshev I
$\frac{1}{2}$	$\frac{1}{2}$	$\sqrt{1-t^2}$	Gauss–Chebyshev II

Code Expert: Radau Quadrature

Notation:

- fixed = "l", "r"
- c = Array of Radau nodes
- b = Array of Radau weights
- r = Interior Gauss-Jacobi nodes
- w = Gauss-Jacobi weights

Coding Tips

- Inspect what scipy.special.roots_jacobi() do
- Obtaining Radau weights Bemerkung 5.5.11
- For order use testing in the same manner as before

Quadrature in 2D

Consider

$$I = \int_c^d \int_a^b f(x,y)\, dx\, dy$$

Step 1: Integrate along x

For fixed y, define $F(y) = \int_a^b f(x,y)\, dx$ approximate it with a quad. rule $\approx \sum_{i=1}^{n_x} w_i^x f(c_i^x, y)$

Step 2: Integrate along y

Now $I = \int_c^d F(y)\, dy$, again use a 1D-rule $\approx \sum_{j=1}^{n_y} w_j^y F(c_j^y)$

⇒ Combining the two gives tensor-product quadrature

Tensor Product formula: $I \approx \sum_{j_1=1}^{n_1} \sum_{j_2=1}^{n_2} w_{j_1}^1 w_{j_2}^2 f(c_{j_1}^1, c_{j_2}^2)$ ← product of 1D-weights

↓ pairing every x-node with every y-node

$c_{j_k}^k$, $w_{j_k}^k$ .. nodes / weight in coordinate direction k

Code Expert: Homogen geladenes Quadrat in kartesischen Koordinaten

Notation:

- f ... 2 variable integrand
- a, b ... integration limits of x
- c, d ... integration limits of y
- N_x, N_y .. number of integration panels

Coding Tips

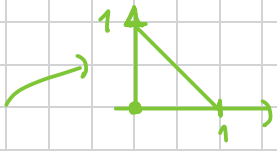
- np.meshgrid(x, y) produces array of $(N_y + 1, N_x + 1)$, so rows corr. to y-vals and columns are x-vals

  ⇒ $\begin{bmatrix} f(x_0,y_0) & f(x_1,y_0) & f(x_2,y_0) \\ f(x_0,y_1) & f(x_1,y_1) & f(x_2,y_1) \end{bmatrix}$, we need some matrix like this!

- must define these, differs for trapezoid and simpson!
- np.sum(..., axis = ?) could help for step 1 of trapezoid2D

From a square to a triangle

$$T_{ref} = \{(x,y) : 0 \le x \le 1 \;,\; 0 \le y \le 1-x\}$$



→ Now as y depend on x, we cannot directly use a tensor-product as we did for the earlier exercise. We need to transform.

Mapping reference triangle to any triangle

T_{ref} has vertices $(0,0)$, $(1,0)$, $(0,1)$ and area $A_{T_{ref}} = 1/2$

→ Suppose another triangle T has vertices V_0, V_1, V_2. We want 3 ref. vertices to map to these vertices.

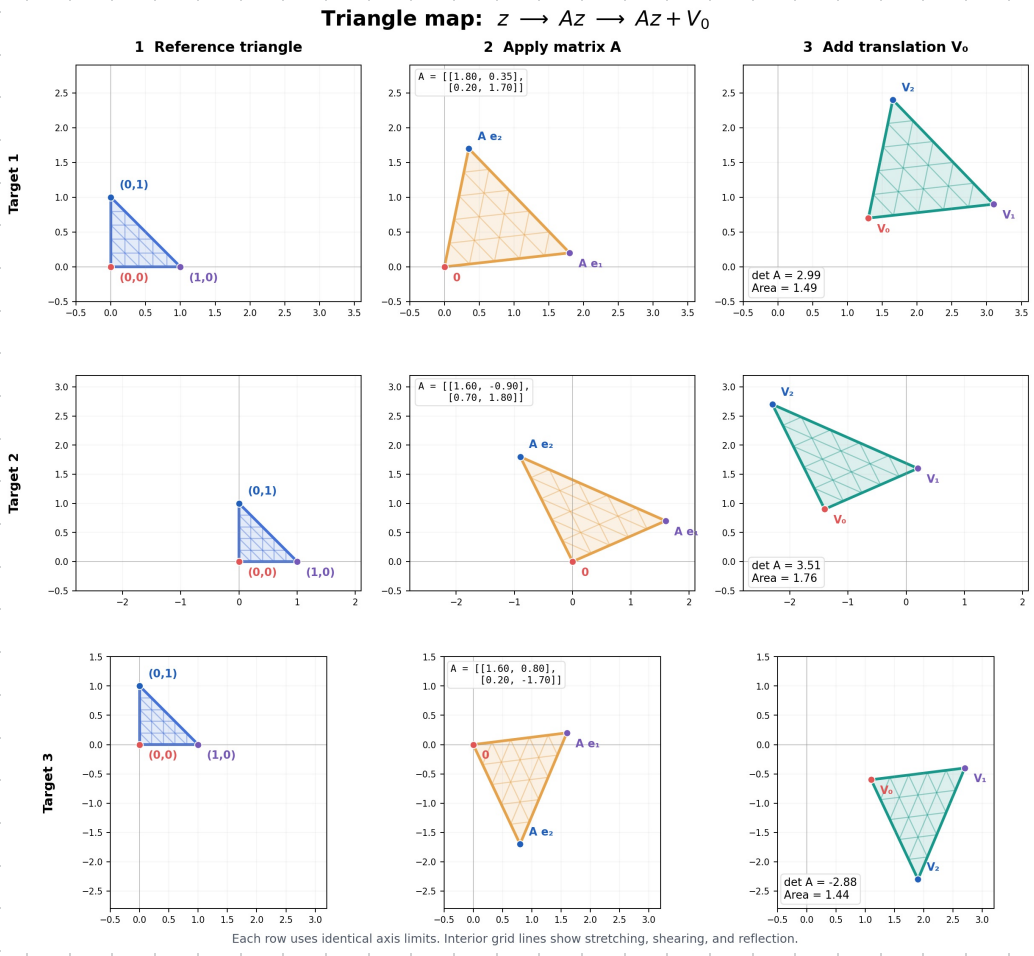
Let $A = \begin{pmatrix} | & | \\ V_1 - V_0 & V_2 - V_0 \\ | & | \end{pmatrix}$ then $x = V_0 + A\hat{x}$ ... here $\hat{x}$ is a reference triangle point.

→ Two columns of A are edges starting at V_0. Multiplication by A stretches or shears T_{ref} into the correct shape and adding V_0 moves it to correct position

$$\int_T f(x)\,dx = |\det A| \int_{T_{ref}} f(V_0 + A\hat{x})\,d\hat{x}$$

→ we will use Quadrature here

accounts for geometric change as A is a transformation



Code expert: Quadraturformeln auf einem Referenzdreieck

Precalculated: self.nodes / self.weight → for reference triangle
self.f → function to integrate

Coding Tips _call_ and getnw

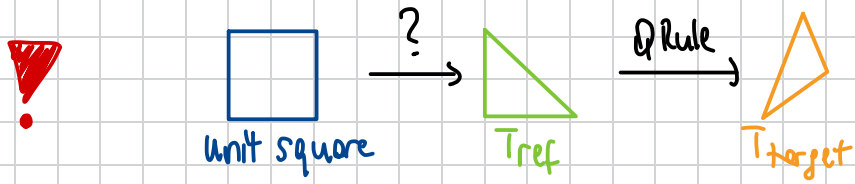
- For _call_ map nodes with transformation above. Evaluate self.f at transformed nodes. Combine function vals and weight, accounting determinant of the transformation.
- getnw should prepare $x = V_0 + f(V_0 + A\hat{x})$ and the transformed weights for physical triangle (target triangle, not the reference we already have them)

Coding Tips - determining poly. exactness

In 1D we tested x^k, in 2D monomials of deg k are $f(x,y) = x^{k_1} y^{k_2}$ where $k = k_1 + k_2$.

Ex.: For degree at most 2 we must test $1, x, y, x^2, xy, y^2$.

⇒ QRule applies a reference quad formula to arb. triangles.



QRule takes a quad. rule def. on reference triangle and transforms it to a a rule for any given triangle

Duffy Transformation

Now we see, how to construct a reference triangle using Gauss-Legendre Quadrature.

→ We alr. know tensor-product quadrature on squares. We therefore transform unit square into a reference triangle.

Use $x = u$, $y = v(1-u)$, $(u,v) \in [0,1]^2$

↳ This works because for u fixed and $0 \le v \le 1$:

$$0 \le y = v(1-u) \le 1-u = 1-x$$

⇒ $$\int_{T_{ref}} f(x,y)\,dA = \int_0^1 \int_0^1 f(c, p(1-c))\,(1-c)\,dp\,dc$$

(1-c) comes from det. of transformation again

↳ Use 2D-Gauss legendre !